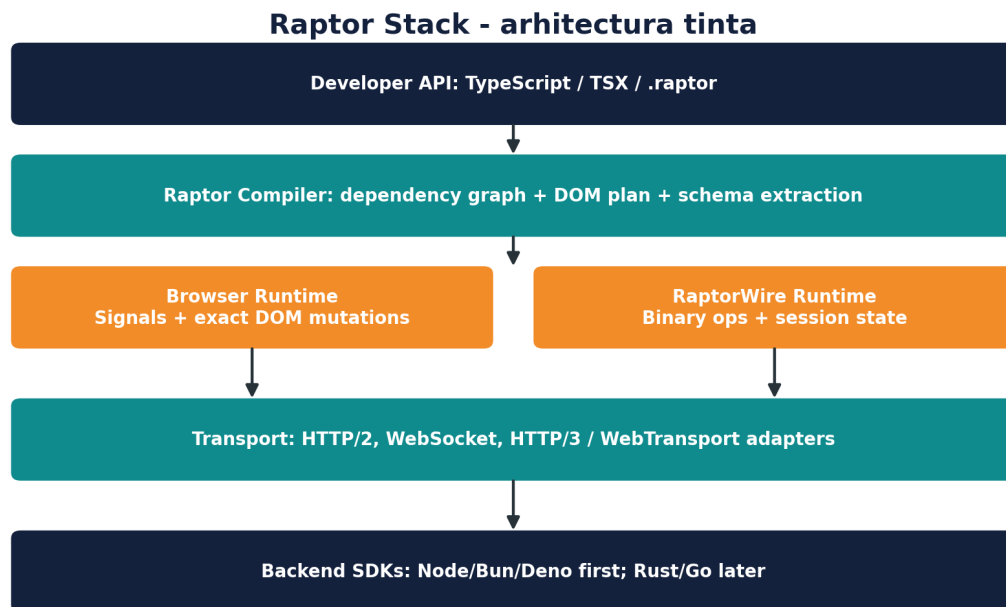


RAPTORJS + RAPTORWIRE

Concept Whitepaper si specificatie preliminara v0.1



Viziunea: compiler-centric UI + protocol binar state-aware + runtime minim.

Document de concept pentru un nou stack web TypeScript, orientat pe performanta, reactivitate fine-grained si sincronizare directa a starii.

21 septembrie 2026

0. Statutul documentului

Important

RaptorJS si RaptorWire sunt nume de lucru. Documentul descrie o arhitectura propusa, nu un produs existent si nu contine rezultate de benchmark masurate. Toate tintele de performanta sunt obiective care trebuie validate experimental.

Scopul acestui whitepaper este sa transforme ideea initiala intr-o baza suficient de concreta pentru prototipare. Sunt definite principiile de design, modelul de reactivitate, rolul compilerului, protocolul binar state-aware, API-uri preliminare, strategia de compatibilitate, securitatea, testarea, roadmap-ul si criteriile prin care proiectul ar putea demonstra un avantaj real.

Diferentierea nu poate fi formulata corect ca "fara Virtual DOM" sau "foloseste signals", deoarece ecosistemul modern are deja framework-uri compiler-centric si sisteme fine-grained. Propunerea devine interesanta prin integrarea end-to-end a compilerului UI cu un protocol de sincronizare a starii care transmite operatii semantice, nu doar obiecte serializate.

Cuprins

Sectiune	Continut
1	1. Rezumat executiv
2	2. Problema pe care incercam sa o rezolvam
3	3. Principii de design
4	4. Peisaj tehnic si diferentiere
5	5. Arhitectura Raptor Stack
6	6. Modelul de executie RaptorJS
7	7. Sistemul de reactivitate
8	8. Compilerul Raptor
9	9. Sintaxa si API-ul developerului
10	10. Componente, lifecycle si ownership
11	11. RaptorWire - obiective
12	12. Modelul de date si schema
13	13. Operatii state-aware si delta updates
14	14. Sesiune, handshake, ACK si resync
15	15. Query, subscriptions si server signals
16	16. Mutatii, tranzactii si optimistic UI
17	17. Transport si conexiuni
18	18. SSR, streaming si reluarea starii
19	19. Routing, formulare si styling
20	20. Offline, cache si sincronizare
21	21. Securitate si limite de incredere
22	22. Observabilitate si DevTools
23	23. Interoperabilitate si adoptie incrementală
24	24. Backend SDK si server runtime
25	25. Performanta - ipoteze si benchmark
26	26. Exemple de aplicatii

27	27. Structura monorepo si pachete
28	28. MVP v0.1
29	29. Roadmap v0.2-v1.0
30	30. Riscuri si decizii deschise
31	31. Specificatie preliminara RaptorWire v0.1
32	32. Referinta API RaptorJS v0.1
33	33. Strategie de testare
34	34. Guvernanta, licenta si comunitate
35	35. Concluzie
36	Anexa A. Exemple complete
37	Anexa B. Glosar
38	Anexa C. Surse si context tehnic

1. Rezumat executiv

RaptorJS este propus ca un framework/compiler TypeScript pentru interfete web in care componentele sunt in primul rand o forma de authoring si analiza statica. In loc ca browserul sa pastreze un model de componenta greu si sa reexecute portiuni largi ale UI-ului la fiecare schimbare, compilerul construiesc in build-time un graf de dependente si un plan de mutatii DOM. La runtime, o schimbare de semnal poate fi propagata direct catre binding-urile afectate.

RaptorWire este protocolul complementar. El nu isi propune sa fie doar un alt encoder binar pentru obiecte. Ideea centrala este ca, dupa sincronizarea initiala, clientul si serverul impartasesc o intelegere a schemelor, obiectelor si versiunilor de stare. Mesajele ulterioare pot descrie operatii compacte precum SET, INCREMENT, APPEND, INSERT, MOVE sau REMOVE. Astfel, reseaua transporta schimbarea semantica, nu o reconstructie repetata a aceluiasi obiect.

Impreuna, cele doua subsisteme urmaresc o cale scurta de la schimbarea de pe server la pixelul afectat in browser: serviciu sau baza de date -> graf reactiv server -> operatie RaptorWire -> semnal client -> binding exact -> mutatie DOM. In sens invers, actiunile utilizatorului pot deveni comenzi tipate care trec prin validare, autorizare si tranzactie, apoi produc un nou set de operatii sincronizate.

Teza proiectului

Avantajul potential nu vine dintr-un singur truc de performanta, ci din eliminarea repetata a muncii redundante intre compiler, runtime, serializare, cache si UI. Daca fiecare strat cunoaste aceeaasi schema si acelasi graf de stare, pot disparea multe conversii intermediare.

- TypeScript-first si integrare buna cu Vite in prima etapa.
- Fara Virtual DOM ca mecanism central; update-uri fine-grained catre binding-uri DOM.
- Runtime de client foarte mic, cu logica mutata spre compiler.
- Protocol binar cu schema negociata si operatii delta/state-aware.
- Server signals, subscriptions si query-uri reactive ca primitive native.
- Fallback la HTTP/JSON pentru interoperabilitate si migrare graduala.
- Instrumentare obligatorie: profiler, wire inspector, reactive graph inspector.
- Benchmark public si reproductibil, nu afirmatii de marketing fara masuratori.

2. Problema pe care incercam sa o rezolvam

2.1 Costul compozit al unei aplicatii moderne

Intr-o aplicatie tipica, o schimbare de date poate traversa mai multe reprezentari: raspuns HTTP, text JSON, obiect JavaScript, cache de request, store, componenta, arbore virtual sau template runtime, apoi DOM. Fiecare strat este justificat individual, dar combinatia poate introduce parsare, alocari, clonari, invalidari si cod de integrare.

Problema nu este ca JSON sau React ar fi "rele". Ambele au avantaje majore: interoperabilitate, ecosistem si model mental familiar. Obiectivul Raptor este sa ofere o cale alternativa pentru aplicatii in care latenta, trafic, update-uri frecvente sau numarul de entitati fac importanta eliminarea muncii redundante.

2.2 Cod de sincronizare repetitiv

Frontend-urile reale contin adesea mecanisme separate pentru fetch, cache, invalidare, WebSocket, reconcilierea raspunsurilor, optimistic updates, retry si resync. Cand serverul si clientul au deja o schema comuna, o parte din acest cod poate fi generata si standardizata.

2.3 Serializare orientata pe document

JSON descrie foarte bine un document. Pentru stari care se schimba des, insa, poate fi inefficient sa retrimitem numele campurilor si portiuni mari ale structurii doar pentru a comunica o mica modificare. Formatele binare existente reduc costul de serializare, dar multe raman orientate pe mesaje complete. RaptorWire propune operatii aplicate asupra unei stari versionate.

2.4 Runtime versus compiler

Cu cat framework-ul poate determina mai mult la build-time - dependente, binding-uri stabile, schema, event handlers si boundaries - cu atat browserul poate executa un runtime mai simplu. Acest transfer nu este gratuit: compilerul devine mai complex si debugging-ul trebuie proiectat foarte bine. Totusi, este o directie potrivita pentru un stack care vrea sa optimizeze end-to-end.

3. Principii de design

Principiu	Consecinta de arhitectura
P1 - Predictibilitate	Un update trebuie sa aiba o cale usor de explicat de la sursa la efect. Evitam magia care face imposibila inspectarea dependintelor.
P2 - Compiler first	Orice informatie stabila la build-time trebuie folosita pentru a reduce munca runtime.
P3 - Fine-grained by default	Schimbarea unui scalar nu ar trebui sa reevalueze o componenta intreaga daca binding-ul poate fi actualizat direct.
P4 - Protocol semantic	Transmiterea unei operatii semantice este preferata retrimiterii unui document cand ambele parti au deja starea de baza.
P5 - Type safety end-to-end	Schema de retea, API-ul server si binding-urile client trebuie sa derive din aceeasi sursa tipata.
P6 - Graceful fallback	Protocolul propriu nu trebuie sa transforme aplicatia intr-o insula. JSON/HTTP si WebSocket standard raman cai de compatibilitate.
P7 - Security before compression	Nicio optimizare nu are prioritate fata de validare, autorizare, limite de resurse si protectie impotriva desincronizarii malitioase.
P8 - Measure everything	Orice afirmatie despre viteza se bazeaza pe benchmark-uri publice, cu cold start, CPU, memorie, bytes si latentia.

4. Peisaj tehnic si diferente

Raptor nu porneste de la premisa ca ecosistemul actual nu are solutii performante. Dimpotriva, mai multe proiecte moderne demonstreaza deja valoarea compilerelor si a reactivitatii fine-grained. De aceea diferentierea trebuie formulata precis.

Tehnologie / familie	Ce demonstreaza deja	Lectie pentru Raptor
React	Model render/commit si reconciliere care aplica doar modificarile DOM necesare.	Nu este suficient sa spunem "React actualizeaza tot DOM-ul"; tinta trebuie sa fie reducerea reevaluarii si a costului de coordonare.
Solid	Fine-grained reactivity si update-uri tintite.	Raptor trebuie sa ofere mai mult decat signals: compiler + protocol + server graph.
Svelte	Compiler puternic si sintaxa reactiva integrata.	Mutarea muncii in build-time este valida, dar developer experience ramane critic.
Angular signals	Tracking granular al utilizarii starii.	Signals devin mainstream; diferentierea nu poate fi simpla lor existenta.
Vue reactivity	Sistem reactiv matur, proxies/ref/computed.	Ergonomia si predictibilitatea API-ului conteaza la fel de mult ca micro-performanta.
Protocol Buffers	Schema tipata, cod generat, payload compact si parsare rapida.	RaptorWire trebuie sa justifice de ce operatiile state-aware adauga valoare peste o serializare binara clasica.
CBOR / MessagePack	Reprezentari binare generale, compacte.	Un format nou fara semantica de stare ar reinventa inutil serializarea.
WebTransport	Transport modern pe HTTP/3, cu streams si datagrams.	RaptorWire trebuie sa fie independent de transport si sa poata profita de WebTransport cand este disponibil.

Diferentierea propusa

RaptorJS + RaptorWire devin interesante daca acelasi compiler poate produce simultan: codul DOM, graful reactiv, manifestul de schema, ID-urile stabile pentru signals/fields si codec-ul de operatii. Astfel front-end-ul si wire protocol-ul nu mai sunt doua sisteme lipite ulterior.

5. Arhitectura Raptor Stack

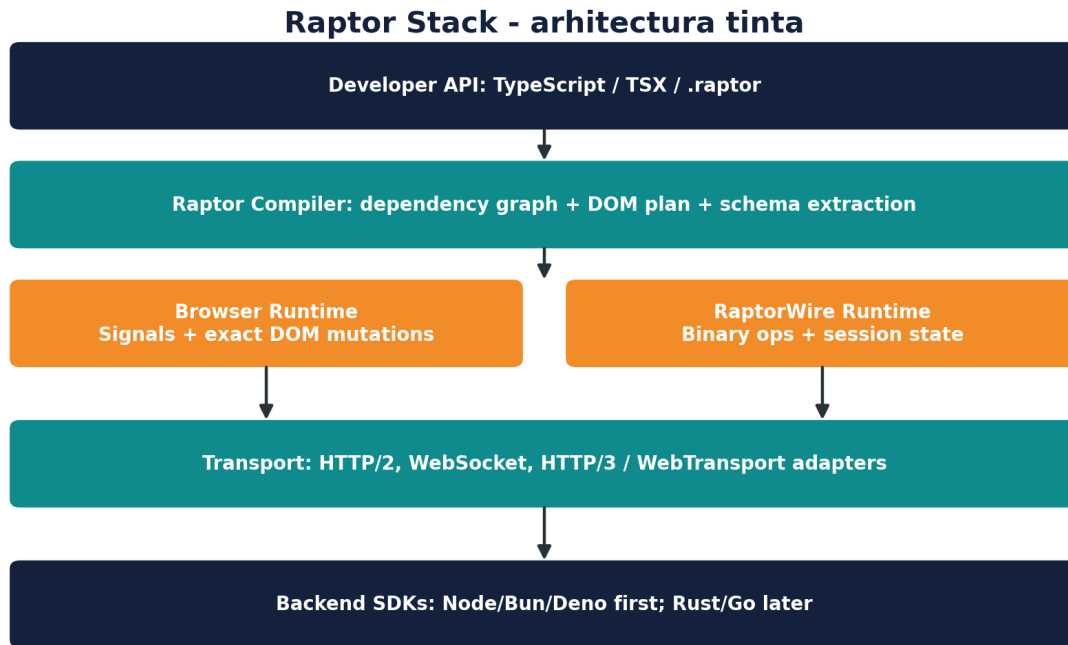
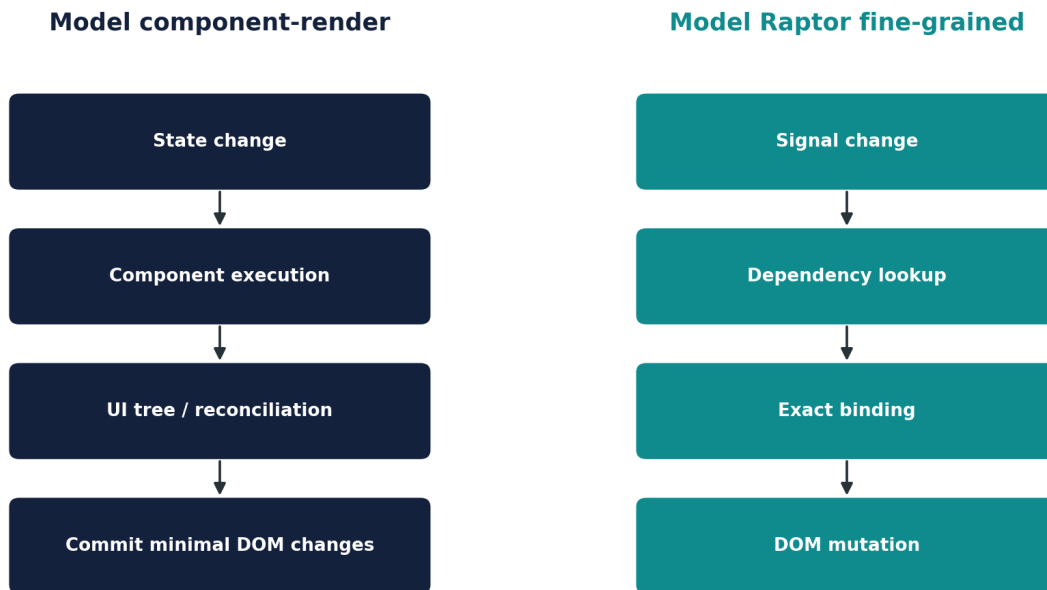


Figura 1. Straturile principale ale stack-ului propus.

Arhitectura este impartita intentionat in straturi independente. RaptorJS poate functiona fara RaptorWire, folosind fetch/JSON. RaptorWire poate avea SDK-uri server-side care nu depind de UI. Integrarea lor produce insa calea optimizata.

Strat	Responsabilitate	Artefacte
Authoring	TS/TSX sau format .raptor; componente, signals, queries.	Fisiere sursa, tipuri, styles.
Compiler	Parse, type analysis, reactive graph, DOM plan, schema extraction, code splitting.	JS/ESM, manifest, source maps, schema hash.
Client runtime	Signals, scheduler, DOM bindings, events, hydration/resume, wire client.	Runtime minim si adaptor.
Wire runtime	Framing, codec, versioning, sessions, op application, ACK/resync.	Binary frames si state table.
Transport	HTTP fallback, WebSocket, WebTransport.	Conexiuni si streams.
Server SDK	Queries, subscriptions, mutations, auth hooks, schema registry.	Node/Bun/Deno initial; alte limbaje ulterior.

6. Modelul de executie RaptorJS



Obiectivul nu este doar "mai puțin JavaScript", ci o cale de actualizare mai directă și predictibilă.

Figura 2. Diferența conceptuală dintre rerender/reconciliation și binding-uri fine-grained.

6.1 Componentele ca unități de authoring

O componentă Raptor este în primul rând o unitate pentru developer: organizează markup, logica, styles și ownership. După compilare, nu este obligatoriu să existe o instanță grea de componentă pentru fiecare nod al UI-ului. Compilerul poate coborî expresiile reactive în bindings individuale.

Exemplu de authoring

```

export component Counter() {
  const count = state(0);

  return (
    <button on:click={() => count.update(v => v + 1)}>
      Clicks: {count}
    </button>
  );
}
  
```

Compilerul poate transforma partea stabilă în creare directă de noduri DOM și poate lega text node-ul numai de semnalul count. La schimbare, runtime-ul nu trebuie să reexecute întreaga funcție de componentă dacă dependențele sunt statice și cunoscute.

Output conceptual, simplificat

```

const b = document.createElement("button");
const t = document.createTextNode("Clicks: 0");
b.append(t);

const count = createSignal(0);
b.addEventListener("click", () => count.set(count.get() + 1));
bindText(t, () => `Clicks: ${count.get()}`);
  
```

6.2 Static regions si dynamic bindings

Template-ul este impartit in regiuni statice si expresii dinamice. Regiunile statice pot fi create o singura data, eventual din template cloning optimizat. Fiecare expresie dinamica devine un binding cu un set de dependente. Compilerul alege strategia in functie de complexitate: text binding, attribute binding, conditional region, keyed list sau dynamic component boundary.

6.3 Scheduler

Update-urile sincrone repetate trebuie batched pentru a evita cascade inutile. Scheduler-ul propus are trei cozi: sync-critical pentru stari care trebuie observate imediat, microtask pentru UI normal si frame pentru update-uri vizuale care pot fi grupate inainte de paint. Semantica trebuie sa fie explicita si testabila, nu dependenta de timing accidental.

7. Sistemul de reactivitate

Nucleul reactiv are patru primitive: state, derived, effect si resource. Pentru date remote apar serverSignal, query si mutation. Graful este dinamic acolo unde expresiile au ramuri conditionale, dar compilerul poate predeclara dependentele statice si poate evita tracking-ul runtime inutil.

Primitiva	Rol	Proprietati
state<T>	Valoare locala mutabila.	get/set/update, equality configurabila, batch.
derived<T>	Valoare calculata.	lazy, memoized, dependency tracking.
effect	Side effects non-DOM sau integration points.	cleanup, scheduling, untracked reads.
resource<T>	Date asincrone.	pending/error/value, cancellation, dedupe.
serverSignal<T>	Valoare sincronizata de server.	version, stale state, reconnect semantics.
query<T>	Colectie/reactive result set.	snapshot + operations, pagination, subscription.
mutation	Comanda server-side tipata.	validation, optimistic policy, transaction id.

```
const first = state("Ada");
const last = state("Lovelace");
const full = derived(() => `${first()} ${last()}`);

effect(() => analytics.track("name", full()));
```

O regula importanta: effects nu trebuie folosite ca mecanism universal pentru derivarea starii. Derivarile pure raman in derived, astfel incat graful sa poata fi optimizat si verificat. Compilerul poate emite warning cand detecteaza un effect care doar scrie intr-un alt state fara I/O sau side effect extern.

8. Compilerul Raptor

Pipeline propus pentru compiler



Output: DOM instructions + tiny runtime calls + schema manifest + wire operation metadata

Figura 3. Pipeline-ul compilerului si artefactele produse.

8.1 Etape

1. Parse TS/TSX si colecteaza informatia de tip disponibila.
2. Identifica componente, signals, derived values, resources, event handlers si boundaries dinamice.
3. Construieste un graf de dependente si marcheaza dependentele statice versus dinamice.
4. Genereaza un DOM plan: noduri statice, binding-uri, liste keyed, conditionale si cleanup scopes.
5. Extrage tipurile care traverseaza reseaua si construieste schema manifest.
6. Aloca ID-uri stabile pentru schema, fields, remote signals si operations, folosind hashing controlat plus registry.
7. Emite JavaScript ESM, source maps, manifest de runtime si optional artefacte SSR/server.

8.2 ID-uri stabile si reproducibilitate

Un protocol state-aware nu poate depinde de ordinea accidentala a proprietatilor. ID-urile de camp trebuie stabile intre build-uri compatibile. Varianta recomandata este un registry explicit generat si versionat in repository. Hashing-ul poate detecta conflicte, dar numerele de wire nu trebuie schimbate automat la fiecare refactor cosmetic.

8.3 Optimizari posibile

- Constant folding si eliminarea binding-urilor care devin statice.
- Hoisting pentru template fragments si event handlers stabili.
- Dead reactive branch elimination cand un feature flag este cunoscut la build.
- Schema tree-shaking: clientul primeste doar tipurile necesare chunk-ului incarcat.
- Codec specialization: encoder/decoder generat pentru schema concreta, evitand reflection generica.
- List specialization pentru keyed arrays si primitive arrays.
- Automatic batching pentru secvente de operatii provenite din aceeasi tranzactie.

8.4 Source maps pentru "cod generat"

Compiler-centric frameworks devin frustrante daca stack trace-urile indica numai cod generat. De aceea fiecare binding, operation si wire field trebuie sa poata fi mapat inapoi la fisierul sursa, linia si expresia originala. DevTools trebuie tratat ca parte a arhitecturii, nu ca proiect ulterior.

9. Sintaxa si API-ul developerului

Pentru adoptie, prima versiune ar trebui sa ramana foarte aproape de TypeScript si JSX. Un DSL complet nou mareste costul de tooling, editor support si invatare. Formatul .raptor poate exista ulterior ca optiune pentru Single File Components, dar MVP-ul poate folosi .tsx.

```
import { component, state, derived } from "@raptor/core";

export const UserCard = component((props: { user: User }) => {
  const expanded = state(false);
  const label = derived(() => expanded() ? "Ascunde" : "Detalii");

  return <article>
    <h2>{props.user.name}</h2>
    <button on:click={() => expanded.update(v => !v)}>{label}</button>
    {#if expanded}
    <p>{props.user.email}</p>
    {/if}
  </article>;
});
```

Sintaxa {#if} este doar una dintre optiuni. Alternativ, se poate folosi JSX normal cu helperi compilabili. Criteriul de decizie este cat de bine poate compilerul distinge o regiune dinamica fara runtime generic si cat de naturala ramane experienta TypeScript.

9.1 Propunere .raptor optionala

```
<script lang="ts">
  const count = state(0);
</script>

<view>
  <button @click="count++">Clicked {count} times</button>
</view>

<style scoped>
  button { padding: 10px 14px; }
</style>
```

Decizie recomandata pentru MVP

Incepe cu TSX si un plugin Vite. Nu construi simultan un limbaj nou, un compiler complet, un protocol si un IDE plugin. DSL-ul poate fi evaluat dupa ce arhitectura de runtime demonstreaza valoare.

10. Componente, lifecycle si ownership

Chiar daca instanta runtime este minima, avem nevoie de ownership pentru cleanup. Fiecare component scope detine effects, subscriptions, event listeners, resources si child scopes. Cand scope-ul este eliminat, cleanup-ul trebuie sa fie determinist.

```
onMount(() => {  
  const observer = new ResizeObserver(onResize);  
  observer.observe(host);  
  return () => observer.disconnect();  
});
```

Lifecycle-ul trebuie redus la evenimente cu semantica clara: onMount, onDispose si optional onVisible/onHidden pentru integrarea cu navigatia si islands. Evitam o lista lunga de hooks care oglindesc fazele interne ale rendererului.

10.1 Error boundaries

O eroare intr-un derived sau event handler nu trebuie sa corupa graful reactiv. Boundary-ul retine ultima stare coerenta, dezactiveaza efectele dependente si poate afisa fallback. Pentru remote operations, o eroare de protocol este separata de o eroare de business.

11. RaptorWire - obiective

RaptorWire este definit ca un protocol de aplicatie peste un transport existent. Nu incerca sa inlocuiasca TCP, QUIC, HTTP sau WebSocket. Se ocupa de framing logic, schema, identificarea starii, operatii, versiuni, subscription lifecycle si resynchronization.

- Payload binar compact, dar inspectabil prin DevTools.
- Zero repeated field names dupa negocierea schemei, in majoritatea fluxurilor.
- Operatii delta aplicate unei stari cunoscute.
- Batching atomic sau ordonat pentru tranzactii.
- Compatibilitate explicita intre versiuni de schema.
- Resync sigur cand clientul pierde operatii sau revine dupa offline.
- Fallback la snapshot complet cand delta devine mai scump sau nesigur.
- Codec generat din schema, fara reflection pe hot path.

Ce NU este RaptorWire

Nu este "JSON dar binar". Daca implementarea v0.1 se limiteaza la serializarea obiectelor cu alte delimitatoare, nu exista suficienta diferentiere fata de Protobuf, CBOR sau MessagePack.

12. Modelul de date si schema

Schema defineste tipurile care pot traversa protocolul si ID-urile wire. Pentru TypeScript, compilerul poate deriva schema dintr-un subset sigur de tipuri, dar tipurile structurale complexe trebuie normalizate. Runtime-ul nu poate transporta direct concepte exclusiv compile-time precum conditional types arbitrare.

Forma canonica interna, generata

```
schema User @id(17) {
  id    : u64    @field(1)
  name  : string @field(2)
  active : bool   @field(3)
  balance : decimal(2) @field(4)
  tags   : string[] @field(5)
}
```

Pe wire, schema ID 17 si field IDs 1..5 inlocuiesc numele textuale. Pentru un obiect persistent, runtime-ul poate atribui si un object handle scurt pe durata sesiunii. Astfel operatiile ulterioare nu trebuie sa retrimita cheia primara completa.

Tip logic	Encoding initial propus	Observatii
bool	1 byte sau bit packing in batch	Simplu si rapid.
unsigned int	varint	Eficient pentru valori mici.
signed int	zig-zag + varint	Reduce costul valorilor negative mici.
float32/64	IEEE fixed width	Fara conversii text.
decimal	scaled integer + scale in schema	Evita erorile de reprezentare pentru bani.
string	UTF-8 length-prefixed	Optional dictionary per session.
bytes	length-prefixed raw	Fara base64.
array	typed sequence	Poate primi op-uri INSERT/MOVE/REMOVE.
object	schema id + fields	Field presence bitmap pentru snapshot-uri compacte.
timestamp	integer epoch + unit in schema	Semantica de timezone trebuie separata.

13. Operatii state-aware si delta updates

Dupa snapshot, serverul poate emite operatii asupra unui object handle, field sau collection. Operatiile trebuie sa fie suficient de putine incat codec-ul sa fie simplu, dar suficient de expresive pentru a evita retransmiterea obiectelor.

Opcode	Semantica	Exemplu
SET	Inlocuieste valoarea unui field.	SET object:17 field:4 value:1532.45
INC	Adauga delta numerica.	INC object:17 field:score +1
APPEND	Adauga la sfarsitul unei colectii.	APPEND messages newMessage
INSERT	Insereaza la index/key.	INSERT list:9 after:key42 item
REMOVE	Sterge element/camp optional.	REMOVE list:9 key42
MOVE	Reordoneaza fara retransmitere.	MOVE key42 after:key9
PATCH	Batch de field updates pe acelasi obiect.	PATCH obj:17 {2=name,3=true}
CLEAR	Goleste colectia.	CLEAR notifications
REPLACE	Snapshot partial/complet cand delta nu mai merita.	REPLACE list:9 snapshot

```
// Stare cunoscuta
user.score = 920

// Wire semantic
INC object=17 field=score delta=+1

// Stare rezultata
user.score = 921
```

INC nu este doar o compresie. Are semantica utila pentru concurenta: serverul poate valida ca operatia este comutativa in anumite contexte sau poate pastra intentia. In alte situatii, SET este mai sigur. Schema poate marca ce opcodes sunt permise pentru fiecare field.

13.1 Delta versus snapshot heuristic

Runtime-ul server trebuie sa compare costul estimat al unui batch de operatii cu costul unui snapshot nou. Daca o lista s-a schimbat masiv sau clientul este prea in urma, REPLACE poate fi mai ieftin si mai robust decat sute de op-uri. Aceasta decizie poate fi automata, pe baza numarului de bytes si a versiunii clientului.

14. Sesiune, handshake, ACK si resync



Figura 4. Exemplu de handshake si trecere la sincronizare incrementală.

Fiecare conexiune stabileste o sesiune logica. Sesiunea are protocol version, capability flags, schema hashes, compression options, server epoch si o tabela de subscription-uri. Pentru reconnect scurt, clientul poate prezenta un resume token si ultima sequence confirmata.

Camp HELLO	Rol
protocolVersion	Versiunea de framing/opcode semantics.
clientBuild	Identifica build-ul pentru diagnostic si compatibilitate.
schemaHashes	Listeaza schemele pe care clientul le are deja.
capabilities	webtransport, datagrams, compression, resume etc.
resumeToken	Optional, pentru reluarea unei sesiuni.
lastAck	Ultima secventa confirmata de client.

14.1 Sequence numbers

Operatiile care modifica aceeasi stare trebuie ordonate. Fiecare stream/subscription poate avea sequence propriu pentru a evita blocarea globala. Daca clientul primeste 102 dupa 100, detecteaza lipsa lui 101 si solicita resync sau retransmitere, in functie de transport si politica.

14.2 Idempotentă

Un batch primit de doua ori nu trebuie aplicat de doua ori. Mesajele de mutatie folosesc transaction/request IDs, iar op batches folosesc sequence si epoch. INC necesita atentie speciala: replay-ul accidental ar dubla efectul, de aceea deduplicarea trebuie facuta inainte de aplicare.

15. Query, subscriptions si server signals

Una dintre cele mai puternice idei ale stack-ului este eliminarea lipiturii dintre fetch si realtime. Un query poate produce snapshot initial, apoi poate ramane subscribed. Clientul vede o colectie reactiva, iar serverul trimite numai operatiile care modifica rezultatul.

```
const users = query(User)
  .where(u => u.active === true)
  .orderBy(u => u.name)
  .limit(100);

<ul>
  {#each users by user.id}
    <li>{user.name}: {user.score}</li>
  {/each}
</ul>
```

Compilerul poate extrage shape-ul de date folosit de UI si poate trimite un projection hint serverului. Totusi, serverul ramane autoritatea: nu accepta arbitrary executable predicates provenite de la client. Query DSL-ul trebuie compilat intr-o reprezentare declarativa validabila.

15.1 Server signals

```
const btcPrice = serverSignal<number>("market.BTC.price");
const onlineUsers = serverSignal<number>("presence.onlineCount");

<header>BTC: {btcPrice} | Online: {onlineUsers}</header>
```

Server signal este optim pentru scalari sau small values cu update frecvent. In wire, signal name-ul textual este folosit doar la bootstrap sau este transformat intr-un signal ID. Update-ul ulterior poate fi doar: [signalId][version][value].

16. Mutatii, tranzactii si optimistic UI

Clientul nu ar trebui sa trimita direct opcodes care modifica autoritatea server. SET/INC primite de la server descriu rezultatul sincronizarii; comenzile client->server sunt mutations tipate, supuse autorizarii si validarii.

```
const renameUser = mutation({
  input: { userId: t.u64, name: t.string.max(80) },
  result: User,
  optimistic: ({ userId, name }, cache) => {
    cache.user(userId).name.set(name);
  }
});

await renameUser({ userId: 42, name: "Cristian" });
```

Serverul raspunde cu confirmare si/sau cu operatiile canonice rezultate. Daca optimistic state difera de rezultatul autoritativ, runtime-ul face reconcile la nivel de fields. Pentru tranzactii complexe, batch-ul server poate fi marcat atomic: UI-ul vede toate modificarile impreuna.

Stare mutation	UI behavior
pending	Optional optimistic patch si indicator local.
committed	Optimistic patch devine confirmat; se aplica eventuale diferente canonice.
rejected	Rollback sau compensating patch; se expune eroarea tipata.
unknown after disconnect	Se interogheaza statusul dupa reconnect folosind transaction id.

17. Transport si conexiuni

RaptorWire nu trebuie legat de un singur transport. MVP-ul poate folosi WebSocket pentru simplitate si suport larg. HTTP request/response ramane fallback pentru query/mutation. WebTransport poate fi un adaptor modern pentru scenarii cu multiple streams sau datagrams, dar nu trebuie sa blocheze lansarea proiectului.

Transport	Avantaj	Dezavantaj / rol
HTTP + binary body	Cel mai simplu fallback; compatibil cu infrastructura existenta.	Nu ofera push persistent de unul singur.
WebSocket	Bidirectional, matur, usor de implementat.	Un singur stream logic; backpressure trebuie gestionat.
WebTransport	Streams multiple si datagrams peste HTTP/3.	Suportul trebuie verificat per target; infrastructura mai complexa.
SSE + HTTP mutation	Simplu pentru server push unidirectional.	Mai putin potrivit pentru protocol binar bidirectional complet.

Un design robust permite aceeasi sesiune logica peste transporturi diferite. Framing-ul RaptorWire nu trebuie sa presupuna ca o citire din socket corespunde unui mesaj complet.

18. SSR, streaming si reluarea starii

SSR are doua obiective: first content rapid si SEO. Dupa HTML initial, runtime-ul trebuie sa lege interactivitatea cu cat mai putina munca. Raptor poate explora doua moduri: hydration clasica simplificata si resume metadata, unde compilerul emite ID-uri suficiente pentru a reconecta event handlers si signals fara reexecutarea intregului arbore.

18.1 Snapshot embedded

Serverul poate include snapshot-ul initial intr-un bloc binar/base64 sau intr-un resource separat. Clientul nu trebuie sa refaca imediat acelasi query dupa load. Schema hash si server epoch insotesc snapshot-ul pentru a preveni aplicarea unui payload incompatibil.

```
<script type="application/raptor-state" data-schema="a91f...">  
  BASE64_BINARY_SNAPSHOT  
</script>
```

In productie ar fi preferabila o reprezentare care evita un overhead mare de base64, de exemplu preload/fetch al unui binary resource. Exemplul de mai sus arata doar mecanismul conceptual.

19. Routing, formulare si styling

19.1 Router

Router-ul trebuie sa fie compiler-aware pentru code splitting si data dependencies. O ruta poate declara query-urile critice, iar compilerul produce chunks si prefetch hints. Navigarea poate incepe incarcarea datelor si a codului in paralel.

```
route("/users/:id", {
  load: ({ id }) => query(User).byId(id),
  component: () => import("./UserPage")
});
```

19.2 Formulare

Forms sunt o sursa mare de boilerplate. O schema de mutation poate genera validare client, mapping de fields si erori server. Validarea client ramane UX, nu securitate; serverul valideaza din nou.

```
const form = createForm(updateProfile);

<form use:form>
  <input field="name" />
  <span>{form.errors.name}</span>
  <button disabled={form.pending}>Salveaza</button>
</form>
```

19.3 Styling

MVP-ul nu trebuie sa inventeze un nou CSS engine. CSS normal, CSS modules si scoped style transform sunt suficiente. Compilerul poate tree-shake styles pentru chunks si poate genera stable scope IDs. Integrarea cu Tailwind-like tooling trebuie sa ramana posibila.

20. Offline, cache si sincronizare

Un protocol bazat pe stare trebuie sa defineasca explicit comportamentul offline. Clientul pastreaza snapshot-uri si un journal de mutations locale. La reconnect, trimite server epoch, subscription versions si mutations neconfirmate. Serverul decide daca poate continua incremental sau cere snapshot.

Caz	Strategie
Offline read-only	Serveste ultimul snapshot si marcheaza valorile stale.
Offline mutations	Journal local cu clientMutationId; replay dupa reconnect.
Server epoch schimbat	Invalideaza resume state si cere snapshot nou.
Schema incompatibila	Blocheaza resume si reincarca build/client assets daca politica o cere.
Conflict business	Mutation rejection sau strategie de merge specifica domeniului.
Lista foarte schimbata	Serverul trimite REPLACE in loc de multe op-uri.

RaptorWire nu trebuie sa pretinda ca rezolva automat conflictele distribuite. Pentru colaborare multi-user avansata pot exista adapters CRDT/OT, dar acestea sunt o problema distincta. Protocolul trebuie doar sa poata transporta operatiile si metadata necesare.

21. Securitate si limite de incredere

Regula centrala

Clientul este neautorizat si necredibil, chiar daca foloseste un protocol binar proprietar. Obscuritatea wire format-ului nu este un mecanism de securitate.

- Autentificare la nivel de sesiune si refresh/revocation explicit.
- Autorizare pentru fiecare query, subscription si mutation, nu doar la handshake.
- Validare de tip, lungime, range si cardinalitate in decoder inainte de alocari mari.
- Limite pentru frame size, nesting, collection length, string bytes si outstanding requests.
- Rate limiting si cost budgets pentru query-uri reactive.
- Protection impotriva schema confusion si downgrade de protocol.
- CSRF relevant pentru fallback HTTP bazat pe cookies; origin checks pentru WebSocket/WebTransport.
- No eval, no executable payload; protocolul transporta date si opcodes finite.
- Fuzz testing obligatoriu pentru decoder si state machine.

21.1 Desincronizare malitioasa sau corupta

Daca un frame refera un object handle inexistent, field nepermis sau sequence invalida, clientul nu incearca sa "ghiceasca". In functie de severitate, solicita resync pentru subscription sau inchide sesiunea. State machine-ul trebuie sa fie fail-closed pentru mesaje imposibile.

22. Observabilitate si DevTools

Un wire format binar fara tooling ar afecta dramatic developer experience. DevTools este parte din MVP, macar sub forma unui inspector web simplu sau Chrome extension ulterior.

Instrument	Ce trebuie sa arate
Reactive graph	Signals, derived nodes, subscribers, cause chain.
DOM bindings	Ce expresie controleaza un text/attribute/list region.
Wire inspector	Frame decoded: schema, opcodes, object handles, bytes raw vs decoded.
Timeline	Mutation -> server processing -> op batch -> signal -> DOM commit.
Profiler	CPU per binding, recomputations, allocations, list churn.
Schema viewer	Versions, hashes, compatibility, field IDs.
Resync diagnostics	De ce s-a pierdut incremental state si cat a costat snapshot-ul.

Fiecare operatie trebuie sa poata avea un tracelD optional in development. Production poate elimina metadata costisitoare sau o poate esantiona.

23. Interoperabilitate si adoptie incrementală

Un framework nou are sanse mici dacă cere rescriere completă. Raptor trebuie să poată intra într-o pagină existentă ca island, custom element sau microfrontend. Similar, o aplicație Raptor trebuie să poată importa biblioteci JS normale.

- Raptor component -> Custom Element pentru integrare externă.
- Wrapper pentru React/Vue components în zone izolate, cu cost declarat.
- fetch() normal permis oriunde; RaptorWire este opt-in.
- JSON endpoint generation pentru clienți care nu înțeleg RaptorWire.
- Schema export către OpenAPI/JSON Schema unde este posibil.
- Protobuf bridge optional pentru backend-uri existente, fără a confunda cele două protocoale.

Strategia de migrare pentru o aplicație React poate începe cu o rută sau un dashboard realtime unde avantajul este măsurabil. Nu are sens să rescriem pagini statice doar pentru a demonstra framework-ul.

24. Backend SDK si server runtime

Prima implementare server ar trebui sa fie TypeScript, pentru a reutiliza schema compiler si a scurta timpul pana la un demo. Node.js si Bun sunt tinte naturale; Deno poate fi suportat prin API-uri standard. Rust/Go devin relevante dupa stabilizarea protocolului.

```
const app = raptorServer({ schema });

app.query("activeUsers", {
  input: t.object({ limit: t.u16 }),
  authorize: ({ user }) => user !== null,
  run: ({ db }, { limit }) =>
    db.users.where({ active: true }).orderBy("name").limit(limit)
});

app.mutation("renameUser", {
  input: RenameUserInput,
  authorize: canEditUser,
  transaction: async ({ db }, input) => {
    return db.users.update(input.userId, { name: input.name });
  }
});
```

Server runtime-ul mentine subscription graphs. Cand sursa de date se schimba, reevalueaza numai query-urile afectate sau foloseste change feeds din DB. Acesta este un domeniu complex; MVP-ul poate incepe cu un in-memory store si un adaptor PostgreSQL simplu, apoi evolua spre incremental query engines.

24.1 Separarea business logic de sync engine

Business logic nu trebuie scrisa in opcodes wire. Serverul produce rezultate canonice, iar sync engine calculeaza snapshot/delta. Aceasta separare permite testarea logicii fara conexiune si schimbarea algoritmului de sincronizare fara a rescrie domeniul.

25. Performanta - ipoteze si benchmark

Fara cifre inventate

Nu trebuie publicata nicio afirmatie de tip "de 3x mai rapid" sau "runtime sub 10 KB" pana cand build-ul real nu confirma. In acest document, asemenea valori sunt tinte, nu rezultate.

Benchmark-ul trebuie sa masoare separat render initial, update CPU, memory, JS transferred, protocol bytes si network latency. Un singur benchmark TodoMVC poate ascunde costurile importante pentru aplicatii mari.

Categorie	Scenariu	Metrici
Initial render	1k si 10k rows.	TTI, CPU, allocations, bundle.
Scalar updates	10k updates pe bindings independente.	updates/sec, p95 commit latency.
Keyed list	insert/remove/move in 10k rows.	CPU, DOM ops, memory.
Realtime feed	100/1000 messages pe secunda.	bytes, parse CPU, dropped frames.
Remote patch	Obiect cu 50 fields, un field schimbat.	wire bytes, decode time, GC.
Reconnect	Client ramas 10s/60s in urma.	resume bytes vs snapshot, convergence time.
SSR/resume	Pagina complexa cu interactive islands.	HTML bytes, client JS, interactive latency.

Comparatia trebuie sa includa configuratii idiomatice pentru fiecare competitor, nu implementari intentionat slabe. Pentru React se folosesc optimizarile normale; pentru Solid/Svelte se foloseste stilul recomandat. Scopul este sa aflam unde arhitectura Raptor chiar castiga, nu sa construim un benchmark de marketing.

25.1 Tinte initiale, nu promisiuni

- Runtime core: tinta sub aproximativ 10 KB gzip pentru functionalitatea de baza, daca arhitectura permite.
- Un scalar remote update trebuie sa poata evita reconstructia unui obiect mare si reevaluarea unui subtree.
- Codec-ul generat trebuie sa evite JSON.parse/stringify pe calea RaptorWire.
- P95 UI commit pentru fluxuri realtime trebuie sa ramana stabil sub batching intens.
- Resync trebuie sa aleaga automat intre replay si snapshot pe baza costului.

26. Exemple de aplicatii

Flux end-to-end: stare, operatii, UI si comenzi

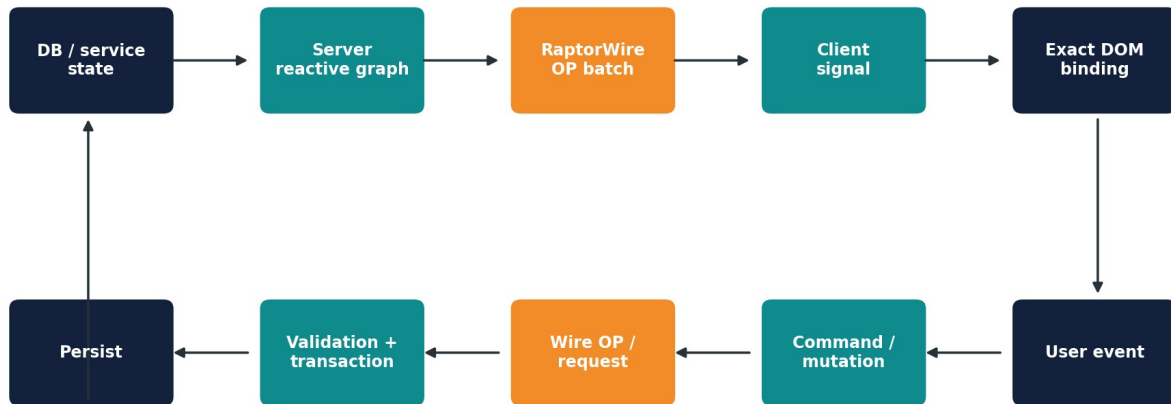


Figura 5. Unificarea fluxului server -> UI si UI -> server.

26.1 Dashboard financiar / operational

Mii de celule se actualizeaza independent. Server signals si opcodes INC/SET pot actualiza direct bindings. Avantajul potential este reducerea serializarii si a rerender-ului pentru zone neafectate.

26.2 Chat

Snapshot initial pentru ultimele mesaje, apoi APPEND pentru mesaje noi, PATCH pentru status delivered/read si REMOVE pentru moderare. Presence poate fi serverSignal sau subscription separata. Reconnect foloseste sequence per channel.

26.3 Magazin / stoc live

Pagina de produs poate primi inventory si price ca signals remote, iar catalogul ca query. O schimbare de stoc nu cere refetch al intregului produs. Mutatia addToCart ramane o comanda autorizata, nu un SET trimis direct de client.

26.4 Aplicatie admin

Tabelele mari beneficiaza de projections, query subscriptions si field-level patches. Formularul poate deriva din mutation schema. Audit trail si trace IDs fac debugging-ul mai usor.

26.5 Continut predominant static

Aici avantajul RaptorWire este mic. Compilerul si SSR pot fi utile, dar complexitatea protocolului nu trebuie activata. Aceasta este o decizie importanta: stack-ul trebuie sa permita modul "RaptorJS simplu" fara persistent connection.

27. Structura monorepo si pachete

```
raptor/
  packages/
    core/      # state, derived, effect, ownership
    compiler/  # TSX transform + reactive graph
    vite/      # Vite plugin
    dom/       # DOM runtime bindings
    router/    # routing + code/data prefetch
    forms/     # schema-driven forms
    wire-core/ # framing/opcodes/state machine
    wire-codec/ # generated encoders/decoders
    wire-client/ # browser session/subscriptions
    server/    # TS server SDK
    devtools-protocol/ # instrumentation format
    devtools/  # inspector UI
    benchmark/ # reproducible benchmark suite
  examples/
    counter/
    realtime-dashboard/
    chat/
  docs/
  rfcs/
  specs/
```

Pachetele wire-core si codec trebuie sa poata fi testate independent de DOM. Similar, core reactivity trebuie sa poata rula in Node pentru teste. Aceasta modularitate previne transformarea framework-ului intr-un monolit dificil de evoluat.

28. MVP v0.1

MVP-ul trebuie sa demonstreze ipoteza arhitecturala cu cel mai mic numar de componente. Nu este nevoie de router complet, SSR perfect, CSS DSL si zece SDK-uri server din prima versiune.

Milestone	Livrabil	Criteriu de acceptare
M1 Core reactivity	state, derived, effect, batch, ownership.	Teste deterministe si graph inspector minimal.
M2 Compiler DOM	TSX -> static DOM + text/attr/event bindings.	Counter, conditionals si keyed list fara component rerender generic.
M3 Vite integration	Dev server, HMR basic, source maps.	Exemplu real ruleaza cu workflow normal Vite.
M4 RaptorWire codec	Schema, snapshot, SET/INC/APPEND/REMOVE, frames.	Roundtrip + fuzz + size benchmarks.
M5 Session sync	Handshake, sequence, ACK, resync.	Reconnect test cu pierdere simulata.
M6 Server signals	serverSignal si simple query subscription.	Dashboard realtime end-to-end.
M7 DevTools basic	Decoded wire + reactive graph.	Developer poate explica de ce s-a actualizat un nod.
M8 Benchmark	Comparatii publice, scriptabile.	Rezultate reproductibile pe CI/local.

Demo-ul corect

Un dashboard cu 5.000-10.000 de valori, schimbari realtime si reconnect este mai convingator pentru aceasta arhitectura decat un Todo simplu. Totusi benchmark-urile trebuie separate de demo-ul vizual.

29. Roadmap v0.2 - v1.0

Faza	Focus	Posibile livrabile
v0.2	Developer experience	HMR robust, diagnostics, component props, context, error boundaries.
v0.3	Data layer	queries, mutations, optimistic updates, cache persistence.
v0.4	SSR	streaming, initial snapshots, hydration/resume experiment.
v0.5	Protocol maturity	schema evolution, compression, dictionaries, backpressure.
v0.6	Ecosystem	router, forms, testing library, adapters.
v0.7	Backend expansion	Rust/Go codec generator sau reference SDK.
v0.8	Tooling	browser extension, profiler, trace viewer.
v0.9	Stability	API freeze candidates, migration tooling, security review.
v1.0	Production contract	semver guarantees, wire compatibility policy, LTS plan.

30. Riscuri si decizii deschise

Risc / intrebare	Impact	Mitigare / experiment
Compiler prea complex	Build times, bugs greu de diagnosticat.	MVP mic; IR simplu; golden tests; source maps.
Protocol proprietar	Ecosistem si debugging mai greu.	Fallback JSON; spec publica; Wireshark/DevTools decoder.
Stateful sessions cresc complexitatea	Reconnect si scaling server dificil.	Resume tokens, external session store optional, stateless snapshot fallback.
Fine-grained graph memory	Multe signals/subscribers pot consuma memorie.	Compact node structures, static bindings, benchmarks de memorie.
Schema evolution	Clienti vechi pot interpreta gresit.	Field IDs stabile, compatibility matrix, version negotiation.
RaptorWire nu bate Protobuf in practica	Complexitate fara castig.	Benchmark direct; daca diferenta e mica, folosim Protobuf pentru snapshots si pastram doar op layer.
Adoptie	Framework nou fara ecosistem.	Interop, Vite, TSX, docs excelente, use-case realtime clar.
SSR/resume devine proiect separat enorm	Intarzie nucleul.	Se amana dupa demo-ul client-side + wire.
Nume indisponibil	Branding/mark conflict.	Verificare separata inainte de public launch; numele ramane provizoriu.

O arhitectura buna trebuie sa permita abandonarea unei ipoteze fara rescriere completa. De exemplu, daca formatul de snapshot custom nu aduce castig fata de Protobuf, wire operation layer poate ramane propriu iar snapshot-ul poate folosi un codec existent.

31. Specificatie preliminara RaptorWire v0.1

Aceasta sectiune este intentionat mai formala. Nu reprezinta o specificatie finala, ci un punct de pornire pentru implementarea reference codec.

31.1 Framing logic

```
Frame :=
  magic?      // doar la stream bootstrap, optional
  protocolVersion // varint
  frameType   // u8
  flags       // varint bitset
  streamId    // varint
  sequence    // varint, daca tipul necesita ordine
  payloadLength // varint
  payload      // bytes
  checksum?   // optional; transportul poate oferi integritate
```

Nu trebuie duplicata inutil integritatea deja garantata de TLS/QUIC. Checksum-ul de aplicatie poate fi util pentru debugging/storage, dar nu este o masura criptografica.

31.2 Frame types

ID propus	Frame	Descriere
0x01	HELLO	Capabilities, protocol, schema hashes, resume.
0x02	WELCOME	Session id/epoch si parametri negociati.
0x03	SCHEMA	Definitii lipsa sau mapping de IDs.
0x10	QUERY	Porneste query/subscription.
0x11	SNAPSHOT	Stare initiala sau replace.
0x12	OPS	Batch incremental.
0x13	ACK	Confirmare sequence.
0x14	RESYNC	Solicitare/raspuns pentru desincronizare.
0x20	MUTATION	Comanda client -> server.
0x21	MUTATION_RESULT	Commit/reject/unknown.
0x30	PING/PONG	Liveness/latency.
0x7F	ERROR	Eroare de protocol/session.

31.3 OPS batch

```

OPS payload :=
  transactionId?
  baseVersion
  resultVersion
  opCount
  repeated Operation

Operation :=
  opcode
  targetKind    // object | collection | signal
  targetHandle
  fieldId?
  value / delta / position ...

```

Target handles sunt session-local. La snapshot, serverul poate spune ca User{id=421} este handle 17. Pana la expirarea sau remaparea handle-ului, update-urile pot folosi 17. Pentru persistenta offline, clientul pastreaza si identity canonica, nu doar handle-ul efemer.

31.4 Dictionary compression

Pentru string-uri repetitive - status names, small enums, tags - sesiunea poate negocia un dictionary. Totusi MVP-ul trebuie sa masoare daca dictionary management merita complexitatea; schema enum IDs pot rezolva deja multe cazuri.

31.5 Compatibility rules

- Field ID nu se reutilizeaza pentru o alta semantica dupa eliminare.
- Adaugarea unui field optional este backwards compatible daca decoderul ignora fields necunoscute in snapshot.
- Schimbarea tipului unui field necesita nou field ID sau schema major version.
- Opcode necunoscut intr-un batch critic produce protocol error; extensiile optional pot avea negotiation flags.
- Schema hash include wire semantics, nu nume de fisier sau formatting.

32. Referinta API RaptorJS v0.1

32.1 Core

```
type State<T> = {
  (): T;
  set(value: T): void;
  update(fn: (value: T) => T): void;
};

function state<T>(initial: T, options?: { equal?: (a:T,b:T)=>boolean }): State<T>;
function derived<T>(compute: () => T): ReadonlySignal<T>;
function effect(fn: () => void | (() => void), options?: EffectOptions): Dispose;
function batch<T>(fn: () => T): T;
function untracked<T>(fn: () => T): T;
```

32.2 Async

```
function resource<T>(loader: (ctx: ResourceContext) => Promise<T>): Resource<T>;

type Resource<T> = {
  value: ReadonlySignal<T | undefined>;
  pending: ReadonlySignal<boolean>;
  error: ReadonlySignal<unknown>;
  reload(): void;
};
```

32.3 Remote

```
function serverSignal<T>(key: RemoteSignalKey<T>): RemoteSignal<T>;
function query<T>(descriptor: QueryDescriptor<T>): RemoteCollection<T>;
function mutation<I,O>(descriptor: MutationDescriptor<I,O>): Mutation<I,O>;
```

32.4 Component utilities

```
function onMount(fn: () => void | Dispose): void;
function onDispose(fn: Dispose): void;
function createContext<T>(defaultValue?: T): Context<T>;
function provide<T>(ctx: Context<T>, value: T): void;
function use<T>(ctx: Context<T>): T;
```

API-ul exact trebuie rafinat prin prototipuri. Obiectivul este sa pastram un set mic de primitive care se compun bine, nu sa reproducem fiecare hook disponibil in alte ecosisteme.

33. Strategie de testare

Framework-ul si protocolul trebuie testate pe mai multe niveluri. Pentru compiler, golden tests verifica input -> IR -> output. Pentru reactive core, property-based tests verifica invarianti. Pentru protocol, fuzzing-ul este esential.

Nivel	Teste
Core reactivity	Dependency add/remove, batching, cleanup, cycles, equality.
Compiler	Golden snapshots, source maps, invalid syntax diagnostics, HMR boundaries.
DOM runtime	Text/attr/property/events, keyed list edge cases, disposal.
Codec	Roundtrip pentru fiecare tip, malformed lengths, integer overflow.
State machine	Dropped/reordered/duplicated frames, reconnect, epoch change.
Security	Fuzz frames, allocation bombs, unauthorized query/mutation, downgrade.
End-to-end	Browser + server + simulated network faults.
Performance	CI trend tracking cu variance thresholds, nu doar un run manual.

33.1 Invarianti de protocol

- Aplicarea unui batch valid exact o data produce resultVersion declarat.
- Un batch replayed nu modifica starea a doua oara.
- Dupa resync, snapshot-ul server si starea client sunt echivalente semantic.
- Decoderul nu alocă memorie necontrolată pe baza unui length field nevalidat.
- Un client cu schema veche fie functioneaza in subset compatibil, fie primește eroare explicită; nu interpretează gresit bytes.

34. Guvernanta, licenta si comunitate

Daca proiectul devine public, spec-ul protocolului trebuie sa fie disponibil independent de implementarea TypeScript. Aceasta permite SDK-uri in alte limbaje si reduce riscul de lock-in. Un proces RFC simplu poate documenta schimbarile majore.

- Repository public cu spec/ si rfcs/.
- Semantic versioning pentru packages; versiune separata pentru wire protocol.
- Compatibility policy publicata inainte de v1.0.
- Reference tests pentru implementari third-party.
- Licenta permisiva pentru framework; numele si trademark-ul gestionate separat daca proiectul creste.
- Benchmark dashboard public pentru a evita regresile si afirmatiile neverificabile.

35. Concluzie

Ideea are potential daca este tratata ca un proiect de sisteme, nu doar ca un nou set de componente UI. Un framework bazat pe compiler si fine-grained reactivity este plauzibil, dar nu este suficient de nou singur. Partea distinctiva este contractul comun intre compiler, schema, server state si UI bindings.

RaptorWire trebuie sa castige dreptul de a exista prin masuratori. Daca o operatie de un singur field poate traversa serverul, reseaua si browserul fara a recrea documente, fara JSON.parse si fara rerender de subtree, iar acest lucru produce castiguri semnificative in scenarii reale, atunci combinatia poate fi puternica. Daca diferenta este mica, arhitectura trebuie simplificata si poate reutiliza serializatoare mature.

Primul obiectiv concret

Construieste un MVP care demonstreaza trei lucruri simultan: (1) binding-uri DOM fine-grained generate de compiler, (2) un stream RaptorWire cu snapshot + operatii delta, (3) un dashboard realtime in care putem masura bytes, CPU, memorie si latenta comparativ cu implementari idiomatice existente.

Anexa A. Exemple complete

A.1 Counter local

```
import { state, derived } from "@raptor/core";

export component Counter() {
  const count = state(0);
  const parity = derived(() => count() % 2 === 0 ? "par" : "impar");

  return <section>
    <h2>{count}</h2>
    <p>Valoare {parity}</p>
    <button on:click={() => count.update(n => n + 1)}>+</button>
  </section>;
}
```

A.2 Dashboard remote

```
const cpu = serverSignal<number>(Signals.cpuUsage);
const memory = serverSignal<number>(Signals.memoryUsage);
const jobs = query(Job).where(j => j.status !== "done").orderBy(j => j.priority);

export component Dashboard() {
  return <main>
    <Metric title="CPU" value={cpu} />
    <Metric title="Memory" value={memory} />
    <ul>
      {#each jobs by job.id}
      <li>{job.name} - {job.progress}%</li>
      {/each}
    </ul>
  </main>;
}
```

A.3 Chat operations

```
SNAPSHOT channel:12 version:800 [messages...last 50]
OPS stream:12 base:800 result:801
  APPEND collection:messages objectHandle:501 {...}
OPS stream:12 base:801 result:802
  PATCH object:501 { status: "delivered" }
OPS stream:12 base:802 result:803
  PATCH object:501 { status: "read" }
```

A.4 Mutation cu optimistic UI

```
const toggleTodo = mutation({
  input: { id: t.u64, done: t.bool },
  optimistic(input, cache) {
    cache.todo(input.id).done.set(input.done);
  }
});

<button on:click={() => toggleTodo({ id: todo.id, done: !todo.done })}>
  {todo.done ? "Reopen" : "Done"}
</button>
```

Anexa B. Glosar

Termen	Definitie
Binding	Legatura compilata intre o expresie reactiva si un efect concret, de obicei un nod/atribut DOM.
Fine-grained reactivity	Model in care dependentele sunt urmarite la nivel de valori/expresii mici, permitand update-uri tintite.
Snapshot	Reprezentarea completa sau partiala a starii la o anumita versiune.
Delta / Operation	Schimbare care poate fi aplicata peste o stare de baza cunoscuta.
Object handle	ID compact, de regula session-local, pentru un obiect deja cunoscut.
Sequence	Numar monoton folosit pentru detectarea frame-urilor lipsa/duplicate.
Epoch	Identificator al generatiei starii server; schimbarea lui poate invalida resume state.
Schema hash	Fingerprint al semantici wire pentru o schema.
Server signal	Signal client a carui valoare autoritativa vine de la server.
Resume	Reluarea sincronizarii incrementale dupa reconnect fara snapshot complet.
Resync	Proces de revenire la o stare coerenta cand istoricul incremental nu mai este valid.
Codec specialization	Encoder/decoder generat pentru o schema concreta in locul unui serializer generic.

Anexa C. Surse si context tehnic

Urmatoarele surse oficiale au fost folosite doar pentru a ancora comparatia cu tehnologii existente si a evita afirmatii simplificate. Designul RaptorJS/RaptorWire din acest document ramane o propunere originala de arhitectura si trebuie validat prin prototipare.

Sursa	URL	Relevanta
React - Render and Commit	https://react.dev/learn/render-and-commit	Documenteaza fazele trigger, render si commit si faptul ca React aplica modificarile DOM necesare.
SolidJS - Fine-grained reactivity	https://docs.solidjs.com/advanced-concepts/fine-grained-reactivity	Describe update-uri tintite bazate pe dependente fine-grained.
Svelte - What are runes?	https://svelte.dev/docs/svelte/what-are-runes	Arata rolul compilerului si al sintaxei reactive in Svelte.
Angular - Signals	https://angular.dev/guide/signals	Describe tracking granular al starii si computed/effects.
Vue - Reactivity Fundamentals	https://vuejs.org/guide/essentials/reactivity-fundamentals	Context pentru modelul reactiv Vue.
Protocol Buffers - Overview	https://protobuf.dev/overview/	Context pentru schema tipata, serializare compacta si cod generat.
RFC 8949 - CBOR	https://www.rfc-editor.org/rfc/rfc8949.html	Standard pentru o reprezentare binara concisa a datelor.
MessagePack	https://msgpack.org/	Context pentru serializare binara generala similara modelului JSON.
MDN - WebTransport API	https://developer.mozilla.org/en-US/docs/Web/API/WebTransport_API	Context pentru transport HTTP/3 cu streams si datagrams.

Nota: disponibilitatea si versiunile tehnologiilor web se schimba. Inainte de o implementare production, compatibilitatea browserelor si versiunile framework-urilor trebuie reverificate.

Raptor Stack - urmatorul pas

BUILD THE SMALLEST THING THAT CAN PROVE THE ARCHITECTURE

1. @raptor/core - signals si ownership
2. @raptor/compiler - TSX -> exact DOM bindings
3. @raptor/wire-core - snapshot + ops + resync
4. realtime-dashboard - benchmark end-to-end